

Studieguide: Att produktionssätta en maskininlärningsmodell

AI – teori och tillämpning, del 1
Kursvecka 4, måndag 7 september 2026

Sammanfattning av lektionen om produktsättning av ML-modeller

Dagens mening

Ett ML-API är ett vanligt API plus en artefakt. API-delen är vanlig webbutveckling. Artefakten är det nya, och nästan alla fallgropar kommer från den.

Innehåll

1 Vad betyder det att produktionssätta en modell?	2
1.1 Situationen som hela dagen handlar om	2
2 Kod kontra artefakt	2
3 Artefaktens fyra frågor	3
4 Fråga 1: Vem är jag? - modellregistret	3
4.1 Problemet	3
4.2 Lösningen: ett modellregister	3
4.3 Så byter man modell – och byter tillbaka	4
4.4 Var artefakten bor	4
4.5 Ska modellen ligga på GitHub?	4
5 Fråga 2: Vad behöver jag? - miljön	5
5.1 Problemet	5
5.2 Skydden	5
5.3 Ladda en gång, inte per request	5
6 Fråga 3: Ser datan rätt ut? - två problem	5
6.1 Problem A: preprocessing följer inte med	6
6.2 Problem B: slutanvändaren vet inte vilken form modellen vill ha	6
7 Fråga 4: Hur vet någon om jag har fel? - tester och övervakning	7
7.1 Det finns inget facit i drift	7
7.2 Vad man testar	7
7.3 Monitoring (utanför kursen, men bra att veta)	7
8 Transportmedel: FastAPI och Streamlit	8
9 Koden i sammanfattning	8
10 Vanliga fallgropar	9

11 Checklista inför projektet	9
12 Ordlista	9
13 Instuderingsfrågor	10

1 Vad betyder det att produktionssätta en modell?

Att produktionssätta något betyder att göra det tillgängligt för användning utanför utvecklarens egen dator, så att andra system eller personer kan använda det utan att utvecklaren sitter bredvid. För en maskininlärningsmodell innebär det att modellen ska gå från att vara ett objekt i en notebook – som bara finns i minnet medan notebooken körs – till något som kan ta emot ny data och svara, om och om igen, utan att någon kör celler för hand.

Ett API är *ett* sätt att göra det. Andra sätt är batch-jobb som kör modellen varje natt, en modell inbäddad i en mobilapp, eller en Streamlit-sida. På lektionen använde vi API (FastAPI) och Streamlit, men principerna är desamma oavsett transportmedel.

1.1 Situationen som hela dagen handlar om

Du har tränat modeller i två veckor. Inte en – kanske femton. Olika algoritmer, olika hyperparametrar, olika features. Du har validerat dem och testkört dem. Nu ska en av dem ut till användarna.

- Vilken av dem ska produktionssättas?
- Om den om tre veckor börjar ge dåliga svar - hur byter vi?

Om du byggde ett vanligt API hade de här frågorna inte funnits. Att de finns beror på att det vi produktionssätter inte är kod, utan en *artefakt*.

2 Kod kontra artefakt

Ett vanligt API (t.ex. kundregister)	Ett ML-API
Logiken är kod som en människa skrivit. Ligger i git; text, små filer, meningsfulla diffar.	Logiken är en artefakt: ett objekt fullt av tal som skapats av träning. Binär, kan vara stor, byts ut helt vid varje omträning. Hör inte hemma i git.
Kan läsas - är den fel kan man hitta felet i koden. Beter sig likadant på alla datorer.	Kan inte läsas. Man kan inte öppna filen och se om den är rätt. Beter sig olika beroende på indata, och beror på vilken version av biblioteken som är installerad.
Har facit: ID 42 ska ge kund 42.	Gissar. I drift finns inget facit.

Att produktionssätta en modell är alltså att produktionssätta ett API *plus* en artefakt. FastAPI, uvicorn, endpoints, tester – det är vanlig webbutveckling. Artefakten är det som gör det till ett ML-problem.

3 Artefaktens fyra frågor

En artefakt som ska fungera utanför din dator ställer fyra frågor. Allt vi skrev under lektionen är ett svar på en av dem.

#	Frågan	Svaret i koden
1	Vem är jag?	Metadata bredvid modellen (metadata.json) eller MLflow-register med version och alias (frivilligt)
2	Vad behöver jag för att fungera?	MLmodel bär sklearn-versionen; kontroll i lifespan; pinnad requirements.txt
3	Ser datan ut som jag förväntar mig?	Preprocessing: Pipeline. Form: signatur i artefakten + Pydantic-kontrakt
4	Hur vet någon om jag har fel?	Kontrakttester, sanity check, /health; monitoring i verkligheten

De följande fyra avsnitten går igenom en fråga i taget.

4 Fråga 1: Vem är jag? - modellregistret

4.1 Problemet

Varje träning i notebooken ger ett accuracy-tal som du ser och sedan skriver över. Efteråt har du en mapp med `model_v3_final_FINAL.joblib` och ingen aning om vilken som var bäst, vilka parametrar som gav den, eller när den tränades. En modellfil utan information om sig själv är en anonym binär blob.

4.2 Lösningen: ett modellregister

Det som behövs är att varje sparad modell bär information om sig själv – version, parametrar, mätvärde, träningstillfälle – och att det finns ett tydligt sätt att peka ut vilken som är i drift. Det kan göras på två sätt:

MLflow är frivilligt i projektet

På lektionen använde vi MLflow, men det är *inte* ett krav. Kravet är att fråga 1 besvaras: modellen ska ha en identitet och det ska gå att byta version på ett kontrollerat sätt. Den enklare vägen är helt godkänd:

- **Manuellt:** spara modellen med `joblib.dump` och skriv en `metadata.json` bredvid med sklearn-version, feature-namn i ordning, klassnamn, accuracy och datum. Versionshantera genom att döpa filerna (`model_v1.joblib`, `model_v2.joblib`) och låta en konstant i API:et peka på den som är i drift.
- **Med MLflow:** samma information, men loggad automatiskt, sökbar i ett webbgränssnitt, och bytet är ett alias istället för en kodändring.

Välj det ni har tid för. Poängen är att förstå *varför* informationen behövs, inte vilket verktyg som skriver ner den.

MLflow är ett open source-verktyg (Apache 2.0, gratis) som vi kör lokalt. Det ger fyra saker:

Run Varje träning loggas med parametrar, mätvärden, tidsstämpel och artefakt. Inget skrivs över.

Registrerad modell Artefakten registreras under ett namn (`iris-classifier`).

Version Varje ny träning under samma namn får nästa versionsnummer: 1, 2, 3 ...

Alias En pekare, t.ex. production, som anger vilken version som är i drift.

```
with mlflow.start_run() as run:
    pipeline.fit(X_train, y_train)
    mlflow.log_params(params)
    mlflow.log_metric("test_accuracy", acc)

signature = infer_signature(X_train, pipeline.predict(X_train))
info = mlflow.sklearn.log_model(
    sk_model=pipeline, name="model", signature=signature,
    registered_model_name="iris-classifier",
    metadata={"target_names": [...]}
)
client = mlflow.MlflowClient()
client.set_registered_model_alias("iris-classifier", "production",
                                info.registered_model_version)
```

4.3 Så byter man modell - och byter tillbaka

API:et laddar inte en filsökväg utan `models:/iris-classifier@production`. Registret bestämmer vilken version det blir.

- **Ny modell:** kör `train.py` igen → version $N+1$, aliaset flyttar dit. Starta om API:et. Ingen kod ändras.
- **Rollback:** blev den nya sämre, flytta aliaset tillbaka:

```
mlflow.MlflowClient().set_registered_model_alias(
    "iris-classifier", "production", 1)
```

Nyckelpoäng

Svaret på *vilken ska produktionssättas* är: den du jämfört och valt i registret. Svaret på *hur byter vi* är: valet är en pekare, inte kod.

4.4 Var artefakten bor

Ett modellregister består av två delar:

- **Databas** - vet *vem* filen är: namn, version, alias, mätvärden, parametrar, och en pekare till filen. Hos oss `mlflow.db` (SQLite); i drift PostgreSQL.
- **Object store** - har *filen*: S3, Azure Blob, Google Cloud Storage. Byggda för stora binära filer. Hos oss spelar mappen `mlruns/` den rollen.

Samma arkitektur i miniatyr. Skalar man upp byts adresserna, inte koden.

4.5 Ska modellen ligga på GitHub?

Principiellt nej. Git är för kod. Artefakten hör hemma i ett register eller en object store. Men lokalt - oavsett om det är en joblib-fil med metadata.json eller MLflows `mlflow.db` och `mlruns/` - ligger artefakten på *din* disk, och en ren klon av repot har den inte. Två acceptabla vägar i projektet:

1. Commita artefakten (`model/` eller `mlruns/` + `mlflow.db`) om den är liten. En genväg - säg att det är en genväg.
2. Commita den inte, men se till att python `train.py` återskapar den från en ren klon. README säger hur. Närmare hur det ser ut på riktigt.

Välj den andra om du kan, och skriv i README vilken du valde och varför.

5 Fråga 2: Vad behöver jag? - miljön

5.1 Problemet

Den sparade artefakten innehåller inte modellen som ett självständigt program. Den innehåller en *beskrivning*: "skapa ett Pipeline-objekt, ett StandardScaler-objekt med de här medelvärdena, ett LogisticRegression-objekt med de här vikterna". När filen laddas skapar Python objekten *från den scikit-learn som är installerad just nu* och fyller på med de sparade talen.

Har scikit-learn en annan version än vid träning kan tre saker hända:

1. Klassen finns inte längre → laddningen kraschar. Det snälla felet.
2. Klassen finns men har ändrat interna attribut → halvfärdigt objekt; predict kanske kraschar, kanske ger fel svar.
3. Klassen laddar fint men beter sig annorlunda → modellen svarar, inte som vid träning, och ingen märker något förrän någon jämför siffror.

Fallgrop

Kod kan inte "ladda tyst fel" – saknas en funktion får du ImportError på rad ett. Artefakter kan.

5.2 Skydden

- **Pinnade versioner** i requirements.txt: scikit-learn==1.8.0, inte bara scikit-learn.
- **Virtuell miljö (venv)** ger isolering – projektets bibliotek påverkar inte andra. Den ger *inte* reproducerbarhet av sig själv; det kräver pinning.
- **Docker** fryser dessutom Python-version och operativsystem ihop med modellen. Modell och miljö blir ett paket.
- **Artefakten bär sina krav.** MLflow skriver sklearn_version, requirements.txt och Python-version in i MLmodel-filen automatiskt.
- **Kontroll vid uppstart.** API:et jämför versionen i artefakten mot den installerade och varnar vid avvikelse.

```
trained_with = info.flavors["sklearn"]["sklearn_version"]
if trained_with != sklearn.__version__:
    warnings.warn(f"Modellen tränades med scikit-learn {trained_with}, "
                  f"men {sklearn.__version__} är installerat.")
```

5.3 Ladda en gång, inte per request

Artefakten laddas i FastAPI:s lifespan: allt före yield körs när servern startar, allt efter körs vid nedstängning. Att ladda inne i /predict skulle läsa filen från disk vid varje anrop – sekunder per anrop för en stor modell. Dessutom: saknas modellen kraschar servern tydligt vid start, istället för att första användaren får ett kryptiskt fel. I Streamlit gör @st.cache_resource samma jobb.

6 Fråga 3: Ser datan rätt ut? - två problem

En modell är en matematisk funktion från en rad tal till ett svar. Under träning såg den rader i en viss form, ordning och skala. Vikterna är meningsfulla *bara* för den formen. Modellen

vet inte att första talet är "sepal length i centimeter". Kommer det in en rad i millimeter, oskalad, eller med kolumnerna i annan ordning, så kraschar modellen inte. Den räknar på och svarar självsäkert. Fel.

Detta kallas **training-serving skew** och är den vanligaste orsaken till att ML fungerar hos utvecklaren men inte i drift. Det är egentligen två separata problem med två separata lösningar.

6.1 Problem A: preprocessing följer inte med

I notebooken låg scalern och modellen i samma fil och kördes i följd. I drift sparas ofta modellen men inte preprocessingen. `StandardScaler` är inte en formel – den räknade ut medelvärde och standardavvikelse på träningsdatan och sparade dem inuti sig. Den är en *inlärdd komponent*, precis som modellen.

Lösning: Pipeline. Preprocessing bakas in i artefakten.

```
pipeline = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=1000)),
])
pipeline.fit(X_train, y_train)    # scaler.fit -> transform -> clf.fit
pipeline.predict(X)              # scaler.transform -> clf.predict
```

Det som sparas är hela kedjan. Skalningen kan inte glömmas. Vilken scaler eller modell man väljer spelar ingen roll för principen.

Dataläckage

Scalern får bara anpassas (fit) på träningsdata. Anpassas den på hela datasetet har den "sett" testdatan innan utvärderingen och resultatet blir för optimistiskt. Pipeline skyddar automatiskt: fit rör bara det som skickas till fit.

6.2 Problem B: slutanvändaren vet inte vilken form modellen vill ha

Den som anropar API:et vet inget om kolumnnamn, ordning eller giltiga intervall.

Lösning 1: signaturen i artefakten. `infer_signature` skriver ner kolumnnamn, ordning och typer i `MLmodel`. API:et läser ordningen *därför* istället för att hårdkoda den.

Lösning 2: kontraktet (Pydantic). Ett schema som dokumenterar och validerar vad som får skickas in och vad som kommer ut.

```
class IrisFeatures(BaseModel):
    sepal_length: float = Field(gt=0, description="cm", examples=[5.1])
    sepal_width: float = Field(gt=0, description="cm", examples=[3.5])
    petal_length: float = Field(gt=0, description="cm", examples=[1.4])
    petal_width: float = Field(gt=0, description="cm", examples=[0.2])
```

FastAPI validerar varje request mot schemat: saknat fält, fel typ eller negativt tal ger HTTP 422 med ett tydligt felmeddelande innan vår kod ens körs. Schemat genererar också dokumentationen på /docs.

Begränsning: form, inte innebörd

Kontraktet skyddar mot fel *form*. Inte fel *innebörd*. Millimeter istället för centimeter är fyra giltiga positiva flyttal – kontraktet är nöjt, signaturen är nöjd, pipelinen skalar, modellen svarar. Fel. Enda skyddet är dokumentation (`description="cm"`) och att anroparen läser den.

7 Fråga 4: Hur vet någon om jag har fel? - tester och övervakning

7.1 Det finns inget facit i drift

Ett kund-API kan testas mot facit. En modell gissar; fanns det facit i drift hade vi inte behövt modellen. Tre saker som är självklara för kod gäller därför inte:

1. **Man kan inte testa att svaret är rätt.** 93 % accuracy är ett träningsresultat; blir det 91 % vid nästa omträning är inget trasigt.
2. **Modellen kan bli sämre utan att något ändras.** Världen förändras – kundbeteende, priser, produkter. Modellen tränades på hur det var och gissar på hur det är. Det kallas **drift**. Ingen vanlig mjukvara har den egenskapen.
3. **Därför måste man övervaka.** Se nedan.

7.2 Vad man testar

Testa	Testa inte
Tjänsten startar och artefakten laddas (/health)	En accuracy-tröskel – hör till träning och utvärdering
Kontraktet hålls: rätt fält och typer i svaret vid giltig input	Exakta sannolikheter – ändras vid varje omträning
Ogiltig input avvisas med 422	Vilken version som ska vara laddad
Sanity check: en känd datapunkt ger förväntat svar	Scikit-learns interna beteende

```
@pytest.fixture(scope="module")
def client():
    with TestClient(app) as c:    # 'with' kor lifespan -> artefakten laddas
        yield c

def test_predict_known_sample(client):
    r = client.post("/predict", json=SETOSA)
    assert r.json()["species"] == "setosa"

def test_predict_rejects_negative_value(client):
    r = client.post("/predict", json={**SETOSA, "sepal_length": -1.0})
    assert r.status_code == 422
```

7.3 Monitoring (utanför kursen, men bra att veta)

I verkliga system övervakar man två saker:

- **Indata:** ser fördelningen av inkommande data fortfarande ut som träningsdatan? Kan mätas direkt. En tidig varning.
- **Utfall:** blev prediktionen rätt? Kan mätas först när facit kommer in, ofta veckor senare. Bekräftelsen.

Poängen: en modell i drift är inte "klar".

8 Transportmedel: FastAPI och Streamlit

Båda laddar samma artefakt från samma register. Fråga 1-3 är identiska oavsett transportmedel.

FastAPI	Streamlit
När ett annat <i>system</i> ska anropa modellen. Kontrakt via Pydantic, automatisk dokumentation.	När en <i>människa</i> ska klicka: demo, internt verktyg, prototyp. Reglagen ger bara giltiga tal – inget kontrakt behövs. Enhetsproblemet finns kvar.
Ladda en gång: lifespan.	Ladda en gång: @st.cache_resource.
Testbart med TestClient.	Kör hela skriptet om vid varje interaktion.

Båda är acceptabla i projektet – men valet ska motiveras.

9 Kodens i sammanfattning

Fil	Roll
train.py	Tränar en Pipeline, loggar run till MLflow, sparar artefakten med signatur, registrerar version, sätter alias production.
app.py	Laddar models:/iris-classifier@production i lifespan, läser feature-ordning och klassnamn från artefakten, kontrollerar version. /health, /predict med Pydantic-kontrakt.
test_app.py	Kontrakttester och sanity check med TestClient.
streamlit_app.py	Samma artefakt, gränssnitt för människor.
requirements.txt	Pinnade versioner.

Kommandon:

```
python train.py                                # registrera version N
mlflow ui --backend-store-uri sqlite:///mlflow.db # webbgränssnitt
uvicorn app:app --reload                        # API -> /docs
pytest -v
streamlit run streamlit_app.py
```

Flödet i app.py:

```
MODEL_URI = "models:/iris-classifier@production"

@asynccontextmanager
async def lifespan(app):
    info = get_model_info(MODEL_URI)                # artefaktens pass
    state["model"] = mlflow.sklearn.load_model(MODEL_URI)
    state["feature_names"] = [c["name"] for c in info.signature.inputs.to_dict()]
    state["target_names"] = info.metadata["target_names"]
    # ... versionskontroll ...
```

```

yield
state.clear()

@app.post("/predict", response_model=Prediction)
def predict(features: IrisFeatures) -> Prediction:
    X = features.to_frame(state["feature_names"]) # artefaktens ordning
    class_index = int(state["model"].predict(X)[0])
    ...

```

10 Vanliga fallgropar

1. **Preprocessing följer inte med modellen.** Spara ett Pipeline-objekt, inte bara estimatorn. Felet är tyst och syns inte på lätt data. (*Fråga 3*)
2. **Versionsberoenden.** Pinna versioner. Artefakten kan ladda tyst fel. (*Fråga 2*)
3. **Feature-ordning.** Läs den från signaturen. Utan MLflow finns den bara på ett ställe i koden och ingenting kontrollerar den. (*Fråga 3*)
4. **Modellen laddas per request.** Ladda en gång vid uppstart. (*Fråga 2*)
5. **Hårdkodade absoluta sökvägar.** Använd relativa sökvägar från projektroten. (*Fråga 2*)
6. **Artefakten saknas på den andra datorn.** Gäller både joblib-filer och MLflow-registret. Se avsnittet om GitHub. (*Fråga 1*)
7. **Glömt with i TestClient.** Då körs inte lifespan och artefakten laddas aldrig. (*Fråga 4*)
8. **Fel enhet i indata.** Passerar alla kontroller. Dokumentera. (*Fråga 3*)

11 Checklista inför projektet

1. Pipeline, inte lös modell. (3)
2. Pinnade versioner i requirements.txt. (2)
3. Modellen har en identitet: version, mätvärde, feature-ordning, sklearn-version – via metadata.json eller MLflow (frivilligt). (1)
4. Kontrakt via Pydantic; minst ett test för lyckad request och ett för fel input. (3, 4)
5. Startar från en ren klon – README säger hur registret återskapas. (2)

12 Ordlista

Artefakt

Resultatet av en träning: ett objekt med inlärda tal. Inte kod.

Modellregister

Databas + lagring som ger artefakter namn, version, alias och metadata. MLflow.

Run

En loggad träningskörning i MLflow med parametrar, mätvärden och artefakt.

Alias

En pekare i registret (t.ex. production) som anger vilken version som är i drift.

Object store

Lagring för stora binära filer (S3, Azure Blob). Där artefakten bor i drift; mlruns/ lokalt.

Serialisering

Att spara ett Python-objekt till fil så att det kan laddas igen. Pickle, joblib, skops.

Pipeline

En kedja av transformationer + estimator som tränas, sparas och laddas som ett objekt.

Signatur

Beskrivning av förväntad in- och utdata, sparad i artefakten av MLflow.

Kontrakt

Definitionen av vad ett API tar emot och returnerar. Pydantic-scheman i FastAPI.

Training-serving skew

När datan i drift inte ser ut som datan vid träning. Modellen svarar ändå, fel.

Dataläckage

När information från testdatan påverkar träningen, t.ex. en scaler anpassad på hela datasetet.

Drift

Att en modell blir sämre över tid för att världen förändras, utan att kod ändrats.

Monitoring

Övervakning av indata och utfall i drift.

Lifespan

FastAPI:s mekanism för kod vid uppstart (före yield) och nedstängning (efter).

Endpoint

En adress i API:et som gör en sak: /health, /predict.

HTTP 422

Statuskod för "din data var ogiltig". Skickas av FastAPI när kontraktet inte uppfylls.

13 Instuderingsfrågor

1. Förklara med egna ord skillnaden mellan att produktionssätta ett vanligt API och ett ML-API.
2. Vilka fyra frågor ställer en artefakt? Ge svaret i dagens kod på var och en.
3. Du har tränat tre versioner av en modell och version 3 visar sig ge dåliga svar i drift. Beskriv steg för steg hur du går tillbaka till version 2 utan att ändra kod.
4. Varför är en StandardScaler en inlärd komponent och inte en formel? Vad händer om den inte följer med modellen till drift?
5. Vad är skillnaden mellan problem A (preprocessing) och problem B (form) i fråga 3, och vilka olika lösningar har de?
6. Ge ett exempel på indata som passerar både Pydantic-kontraktet och signaturen men ändå ger fel svar. Varför kan inget test fånga det?
7. Varför testar vi inte att accuracy ligger över en viss tröskel i test_app.py?
8. Vad är drift, och varför är det ett problem som vanlig mjukvara inte har?
9. Ska modellfilen ligga i git? Motivera, och beskriv de två acceptabla vägarna i projektet.
10. Vad består ett modellregister av, och vilken roll spelar mlflow.db respektive mlruns/?
11. Varför laddas artefakten i lifespan och inte inne i /predict?
12. Din projektmodell är en regressor. Vad måste ändras i Prediction-schemat och varför?